

# Spark Shuffling 최적화

다양한 Partition 처리 방식

Adaptive Query Execution (Skew Join은 다음 장)

# Contents

1. Repartition and Coalesce
2. AQE (Adaptive Query Execution)

# Repartition and Coalesce

파티션의 수를 조절하는 방법에 대해 알아보고 DataFrame  
힌트에 대해서도 알아보자

## ◆ Repartition을 하는 이유

- ❖ 전체적으로 파티션의 수를 늘려 병렬성을 증가시키기 위해
- ❖ 굉장히 큰 파티션이나 **Skew** 파티션의 크기를 조절하기 위해서
- ❖ 파티션을 분석 패턴에 맞게 재분배 (**Write once, read many**)
  - 어떤 **DataFrame**을 특정 컬럼 기준으로 그룹핑을 하거나 필터링을 자주 하는 경우
    - 미리 그 컬럼 기준으로 저장해두었다면 그게 **Bucketing**

## ◆ Repartition 방식

### ❖ 2가지 방식이 존재

- repartition
- repartitionByRange

### ❖ **Shuffling**이 발생함. 분명한 이유를 가지고 **Repartition**을 사용해야함

- 많은 경우 repartition이 별 이유없이 사용되어 오히려 시간과 비용 증가
  - 비슷하게 불필요한 counting과 distinct counting과 duplicate제거가 비용 발생!

### ❖ Column이 사용되면 균등한 파티션 크기를 보장할 수 없음

### ❖ 파티션의 수를 줄이는 용도로는 사용불가

- 줄이는 경우에는 Coalesce 사용

## ◆ repartition(numPartitions, \*cols)

### ❖ Hash 기반 Partitioning

- repartition(5)
- repartition(5, "city")
- repartition(5, "city", "zipcode")
- repartition("city")
- repartition("city", "zipcode")

## ◆ `repartitionByRange(numPartitions, *cols)`

- ❖ 지정된 컬럼 값의 범위를 기준으로 파티션을 나누는 방식
- ❖ 데이터 샘플링 기반으로 파티션을 나누기에 결과가 매번 다를 수 있음
  - Nondeterministic
- ❖ 사용법 자체는 앞서 `repartition`과 동일

## ◆ Coalesce가 필요한 경우

- ❖ 파티션의 수를 줄이는 용도 (늘리지 않음)
- ❖ **Shuffling**이 발생시키지 않고 로컬 파티션들을 머지함
  - 따라서 **Skew** 파티션을 만들어낼 수 있음
- ❖ **Column**이 사용되며 균등한 파티션 크기를 보장할 수 없음



## ◆ DataFrame 관련한 힌트들

- ❖ Spark SQL Optimizer에게 Execution plan을 만듬에 있어 특정한 방식을 사용하도록 제안 (최적화된 방식을 변경하기 위해 사용)
- ❖ 두 종류의 힌트들이 존재
  - Partitioning 관련 힌트들
  - Join 관련 힌트들

## ◆ DataFrame Partitioning 관련한 힌트들

❖ COALESCE

❖ REPARTITION

❖ REPARTITION\_BY\_RANGE

❖ REBALANCE

- DataFrame을 테이블로 저장할 때 아주 유용
- 파일의 크기를 최대한 비슷하게 만들어서 저장 (AQE 필요)

```
df1.join(df2, "id", "inner").hint("COALESCE", 3)
```

## ◆ DataFrame Join 관련한 힌트들

### ❖ BROADCAST, BROADCASTJOIN, MAPJOIN

- Broadcast Join 사용 제안

### ❖ MERGE, SHUFFLE\_MERGE, MERGEJOIN

- Shuffle Merge Join 사용 제안
- Spark의 기본 조인 전략

### ❖ SHUFFLE\_HASH

- Shuffle Hash Join 사용 제안
- Full Outer Join에는 사용 불가

### ❖ SHUFFLE\_REPLICATE\_NL

- Shuffle-and-replicate (Cross Join) Join 사용 제안



여러 개가 동시에  
사용될 경우 아래로  
갈수록 우선순위가  
낮아짐

```
SELECT /*+ MERGE(df2) */ *  
FROM df1 JOIN df2 ON df1.order_month = df2.year_month
```

## ◆ DataFrame 힌트 사용법

### ❖ SPARK SQL

- `/*+ hint [, ... ] */`
- 예 1) `SELECT /*+ REPARTITION(3) */ * FROM TABLE`
- 예 2) `SELECT /*+ BROADCAST(table1) */ * FROM table1 JOIN table2 ON table1.key = table2.key`

### ❖ DataFrame API

- `.hint` 메소드 사용
- 예 1) `join_df = df1.join(df2, "id", "inner").hint("COALESCE", 3)`
- 예 2) `join_df = df1.join(df2.hint("broadcast"), "id", "inner").hint("COALESCE", 3)`

# AQE (Adaptive Query Execution)

Spark 3.2부터 기본으로 사용되는 최적화 테크닉!

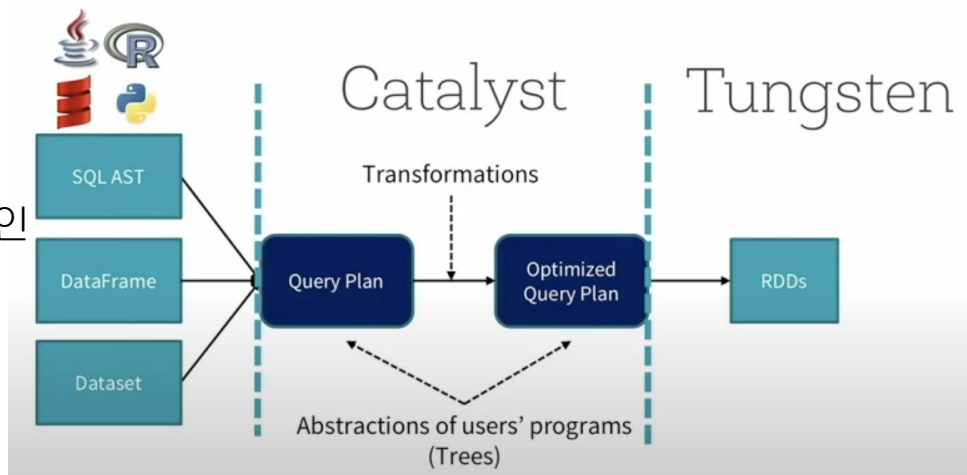
## ◆ Spark Optimization 역사

### ❖ Spark 1.x: Catalyst Optimizer와 Tungsten Project

- 전자는 규칙기반 최적화 수행 (Predicate pushdown, projection pushdown)
- 후자는 기본적으로 JVM 문제없이 코드 최적화를 하려는 것 (GC를 피하기 위해 직접 Off Heap 메모리 관리 수행)

### ❖ Spark 2.x: CBO (Cost-Based Optimizer)

- 데이터프레임 통계정보 이용해 효율적인 execution plan 생성
  - 전체 크기, 레코드 수, 컬럼별 특성 (최소/최대/히스토그램 등등)



Source: Databricks

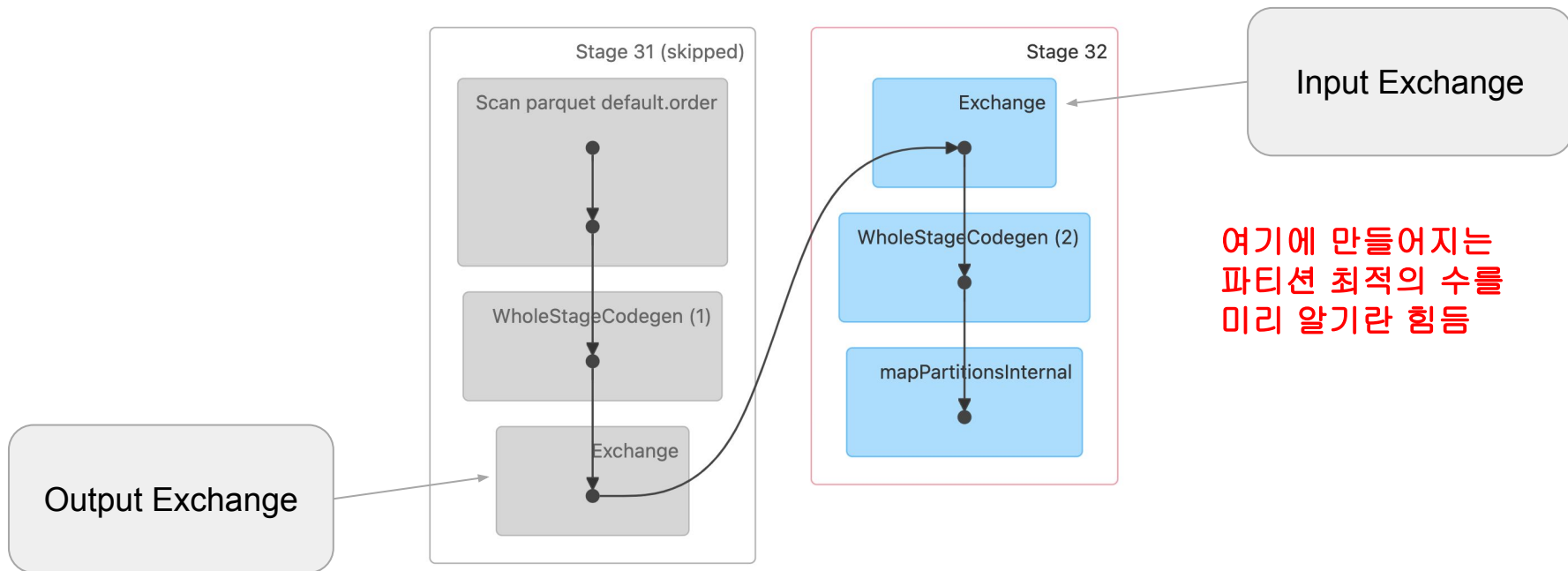
## ◆ AQE (Adaptive Query Execution) 이전 세상 (1)

| item_id | price | order_date          | sku        | destination | factory | order_month |
|---------|-------|---------------------|------------|-------------|---------|-------------|
| 3331590 | 98.0  | 2022-11-01 00:00:00 | PRODUCT-12 | SEOUL       | JEJU    | 2022-11     |
| 2316626 | 27.0  | 2022-11-04 00:00:00 | PRODUCT-3  | SEOUL       | JEJU    | 2022-11     |
| 3331591 | 98.0  | 2022-11-01 00:00:00 | PRODUCT-9  | SEOUL       | BUSAN   | 2022-11     |
| 2316627 | nan   | 2022-11-30 00:00:00 | PRODUCT-97 | SEOUL       | BUSAN   | 2022-11     |
| 3331592 | 22.0  | 2022-11-01 00:00:00 | PRODUCT-5  | SEOUL       | JEJU    | 2022-11     |

```
SELECT sku, SUM(price) sales
FROM order
GROUP BY sku;
```

## ◆ AQE 이전 세상 (2)

- ❖ 앞서 GROUP BY 쿼리는 2개의 stage를 만들어냄
- ❖ spark.sql.shuffle.partitions 값에 의해 Shuffling 후 Partition 수 결정





## ◆ spark.sql.shuffle.partitions

- ❖ 이 변수 하나로 다양한 상황의 **shuffling**을 해결하기는 쉽지 않음
  - MapReduce 세상에서 `mapreduce.job.reduces`와 동일
- ❖ 적은 수의 **Partition**은 병렬성을 낮추고 **OOM**과 **disk spill**의 가능성을 높임
- ❖ 많은 수의 **Partition**은 **task scheduler**와 **task** 생성과 관련된 오버헤드가 생기며 너무 흔한 네트워크 **I/O** 요청으로 병목 초래

만약에 Spark Engine Optimizer가 알아서 **Partition**의 수를 결정할 수 있다면?

## ◆ 앞서 이슈를 해결하려면

- ❖ 대용량 데이터베이스 분야에서는 이 문제는 잘 연구된 문제
- ❖ Intel Big Data 팀이 프로토타입 개발 후 Databricks와 협업 (Spark 3.0)
- ❖ 기본적인 아이디어는 **parsing time** 최적화와 **runtime** 최적화의 병행
  - Parsing time 정보로 선택된 **physical plan**과 코드 최적화만으로는 불충분
  - 특히 **UDF**가 많이 사용되는 경우 이 문제는 더 심각해짐

## ◆ AQE란?

❖ “Dynamic query optimization that happens in the **middle** of query execution based on **runtime statistics**”

- AQE base all optimization decisions on accurate runtime statistics

그러면 언제 이런 실행시간 통계 정보를 뽑고 최적화 방식에 변경을 줄 수 있는 최적의 시점인가?

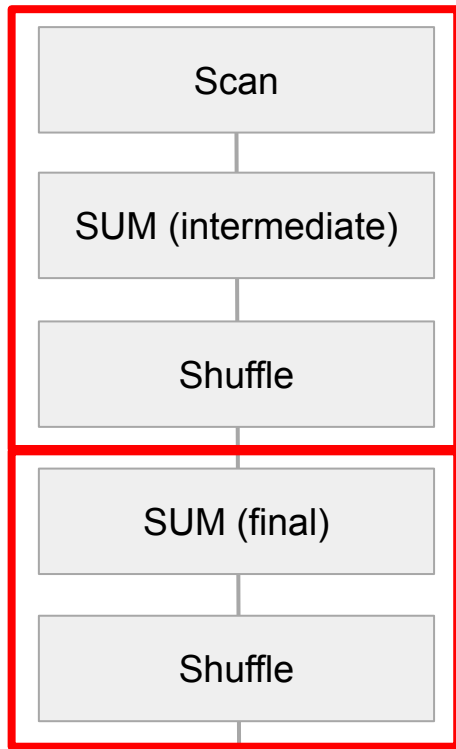
Query -> Job -> **Stage** -> Task

## ◆ Stage가 가장 좋은 최적화 방식 변경 포인트

- ❖ Shuffling/Broadcasting이 Job을 Stage들로 나눴다
- ❖ 또한 이 때 중간 결과들이 materialize됨
  - 따라서 가장 좋은 시점이며 또한 Partition의 수와 크기 정보들도 알 수 있음

```
SELECT sku, SUM(price) sales
FROM order
GROUP BY sku;
```

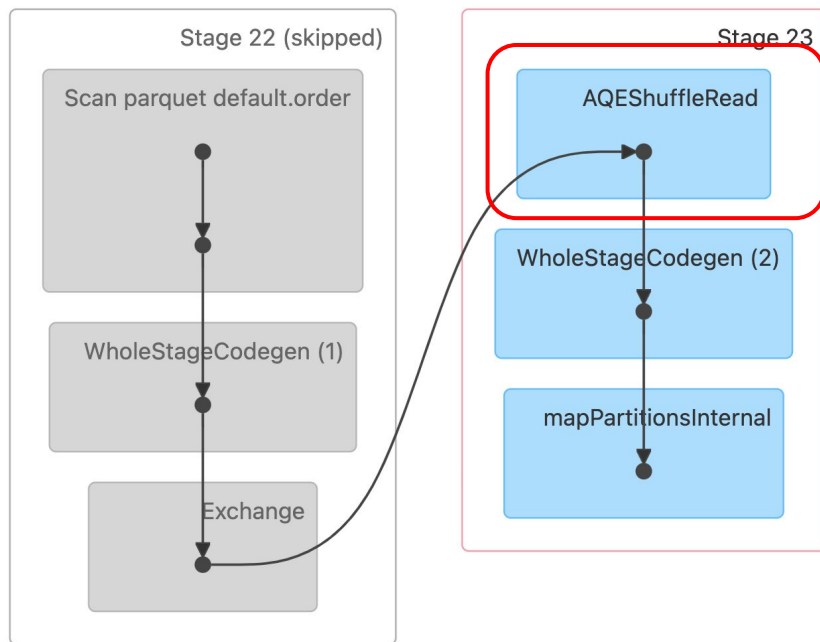
Stage 0



Stage 1

## ◆ AQE 이후 세상

- ❖ 앞서 GROUP BY 쿼리 2번째 Stage 시작부에 AQEShuffleRead가 사용



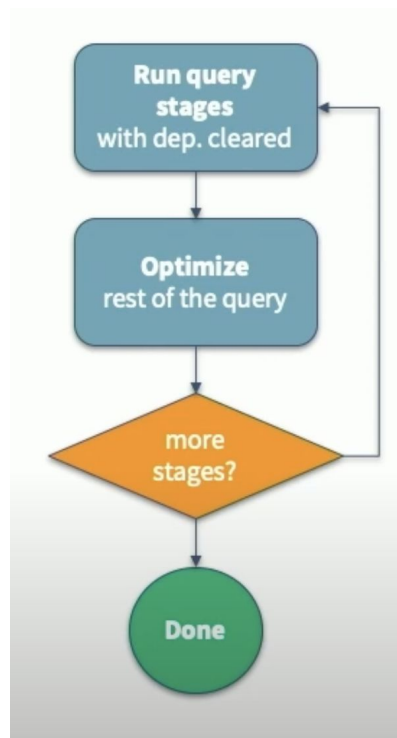
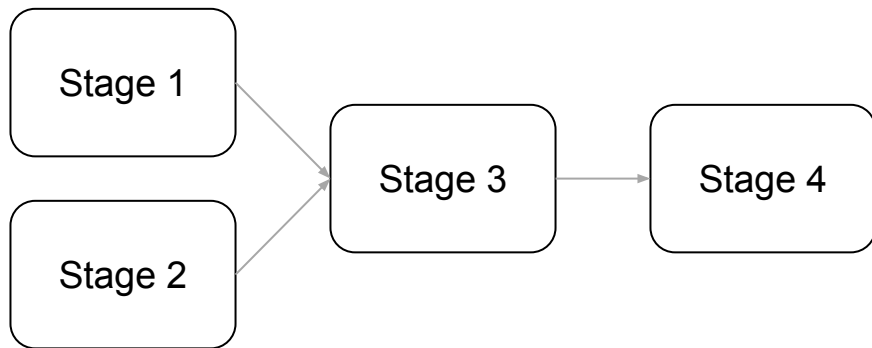
## ◆ AQE가 필요한 경우들

- ❖ Dynamically coalescing (Post) shuffle partitions (Spark 3)
- ❖ Dynamically switching join strategies (Spark 3.2)
- ❖ Dynamically optimizing skew joins (Spark 3)

# Dynamically coalescing (Post) shuffle partitions

## ◆ AQE의 동작 방식

- Stage DAG를 순차적으로 실행
- 매번 새로운 최적화 기회가 있는지 조사
  - 필요하면 다시 실행하거나 쿼리 플랜 변경



Source: Databricks

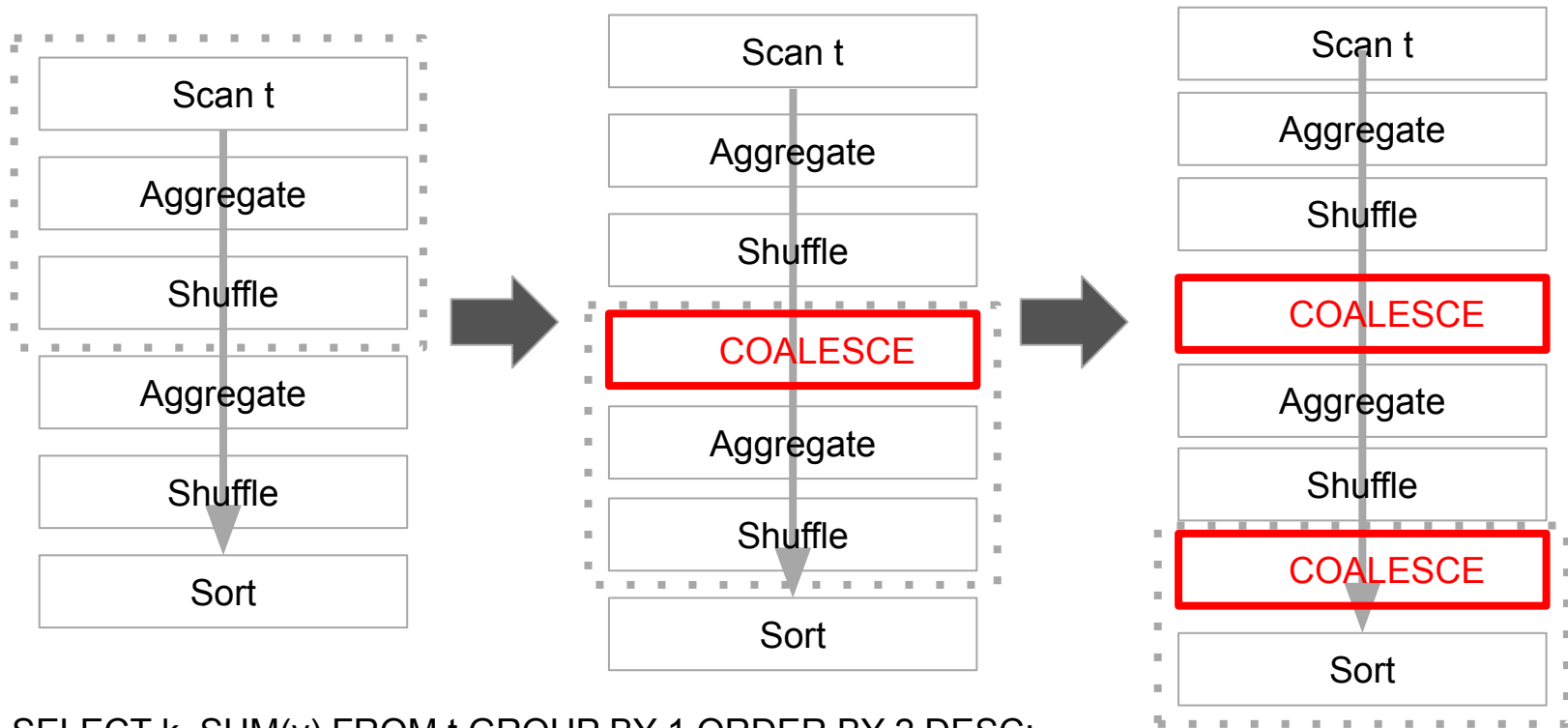


## ◆ Dynamically coalescing shuffle partitions은 무엇인가?

### ❖ 먼저 왜 필요한가?

- 적당한 파티션의 크기와 수는 성능에 지대한 영향을 끼침
- 너무 많은 수의 작은 파티션
  - 스케줄러 오버헤드
  - 태스크 준비 오버헤드
  - 비효율적인 I/O (파일시스템/네트워크)
- 적은 수 큰 파티션
  - GC 악몽 (OOM = Out Of Memory)
  - Disk Spill
- `spark.sql.shuffle.partitions`라는 하나의 변수로는 불충분

## ◆ Dynamically coalescing shuffle partitions 동작방식



SELECT k, SUM(v) FROM t GROUP BY 1 ORDER BY 2 DESC;

## ◆ Dynamically coalescing shuffle partitions 관련 변수들

| 환경 변수 이름                                                               | 기본 값        | 설명                                                                                                                                           |
|------------------------------------------------------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>spark.sql.adaptive.coalescePartitions.enabled</code>             | True        | <code>spark.sql.adaptive.enabled</code> 도 true인 경우 셔플 후 파티션 수를 동적으로 줄이며 파티션의 크기는 아래 변수 ( <code>advisoryPartitionSizeInBytes</code> )로 맞추려 시도 |
| <code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code>           | 64MB        | 셔플링 후 파티션 수를 줄일 때 목표로 하는 파티션의 크기                                                                                                             |
| <code>spark.sql.adaptive.coalescePartitions.parallelismFirst</code>    | <b>True</b> | 이 값이 true이면 병렬성 보장을 위해 위의 목표 크기가 무시되고 아래 <code>minPartitionSize</code> 만 보장                                                                  |
| <code>spark.sql.adaptive.coalescePartitions.initialPartitionNum</code> | 없음          | Coalescing 전의 파티션 수. 없으면 <code>spark.sql.shuffle.partitions</code> 로 설정                                                                      |

## ◆ Dynamically coalescing shuffle partitions 동작방식

### ❖ AQE의 해법은 다음과 같음

- 내부적으로 많은 수의 파티션을 일부러 생성
  - `spark.sql.adaptive.coalescePartitions.initialPartitionNum` (200)
- 매 Stage가 종료될 때 필요하다면 자동으로 Coalesce 수행
  - `spark.sql.adaptive.coalescePartitions.enabled`
- 설정에 따라 파티션의 크기는 최소 크기 혹은 목표 크기를 맞추려 동작
  - `spark.sql.adaptive.advisoryPartitionSizeInBytes`
  - `spark.sql.adaptive.coalescePartitions.minPartitionSize`
  - 무엇을 쓸지는 `spark.sql.adaptive.coalescePartitions.parallelismFirst`에 의해 결정

## ◆ Dynamically coalescing shuffle partitions 그림 (1)

### ❖ Coalescing이 없는 Shuffle

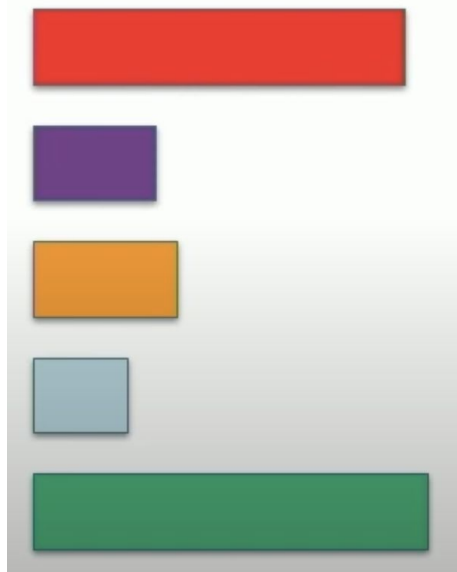


2개의 파티션이 입력이 들어감



Shuffling

`spark.sql.shuffle.partitions = 5`



5개의 파티션 생성

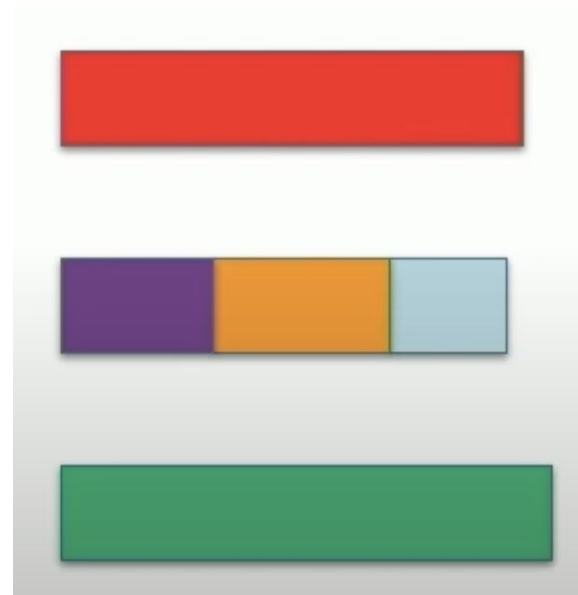
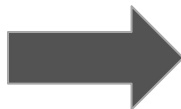
Source: Databricks

## ◆ Dynamically coalescing shuffle partitions 그림 (2)

### ❖ AQE Coalescing이 적용된 경우



2개의 파티션이 입력이 들어감



3개의 파티션 생성

Source: Databricks

## ◆ AQE 데모 환경

- ❖ 2개 테이블을 만들어서 3가지 AQE 기능을 테스트하는데 사용
- ❖ 테이블 1번 (**items**): 일종의 **dimension** 테이블
  - 30,000,000개의 레코드 존재
  - id와 price 필드로 구성
- ❖ 테이블 2번 (**sales**): 일종의 **fact** 테이블
  - 1,000,000,000개의 레코드 존재
  - item\_id와 quantity와 date 필드로 구성
- ❖ 위 두 개의 테이블을 Spark SQL를 사용해서 생성



# Dynamically switching join strategies



## ◆ Dynamically switching join strategies는 무엇인가?

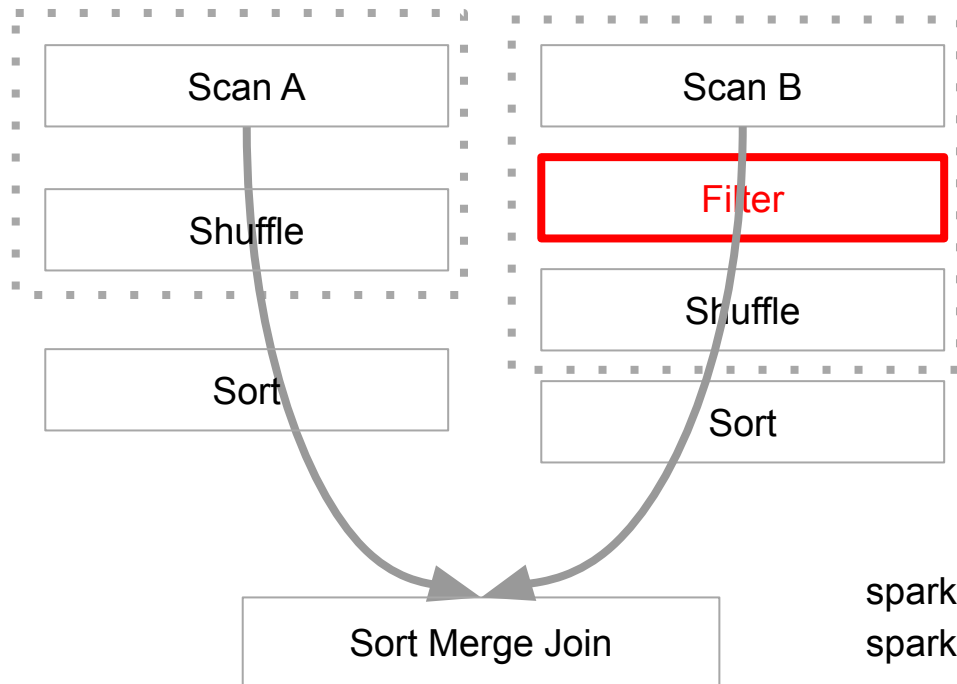
### ❖ 먼저 왜 필요한가?

- Static Query Plan이 여러 이유로 BHJ (Broadcast Hash Join) 기회를 놓친 경우
  - 조인대상 DataFrame들에 대한 통계 정보 부족 (필터링 등)
  - UDF가 사용된 경우

### ❖ AQE의 해법

- Runtime 통계정보를 바탕으로 조인 전략을 변경
  - 이는 **stage**들이 끝나고 조인되기 전에 다시 쿼리 플래닝을 수행
- 아래 두 가지 옵션이 존재
  - **Broadcast Join** (추천되며 우선순위를 가짐)
  - Shuffle Hash Join

## ◆ Dynamically switching join strategies 동작방식



1. Leaf stage 실행
2. 결과를 보고 다시 query planning (최적화)
  - a. B가 필터링 후에 Broadcast 필터링 밑으로 내려가는 경우 B로 Broadcast 조인 수행
3. Static query planning 보다는 성능이 떨어짐 (하지만 여전히 도움이 됨)

`spark.sql.adaptive.autoBroadcastJoinThreshold`  
`spark.sql.autoBroadcastJoinThreshold`

## ◆ Dynamically switching join 관련 환경 변수들 (Spark 3.3.1)

| 환경 변수 이름                                                    | 기본 값 | 설명                                                                                                                                                                                                                          |
|-------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| spark.sql.join.preferSortMergeJoin                          | True | 데이터프레임 조인시 <b>Sort Merge Join</b> 을 기본으로 사용 여부. 항상 <b>Sort Merge Join</b> 을 쓴다는 의미는 아님                                                                                                                                      |
| spark.sql.adaptive.<br>maxShuffledHashJoinLocalMapThreshold | 0    | <b>Hash Join</b> 시 파티션 별로 해시맵 생성에 사용가능한 최대 크기 지정. 이 값이 <b>spark.sql.adaptive.advisorPartitionSizeInBytes</b> 보다 크고 모든 파티션 크기가 이 값보다 작다면 <b>Has Join</b> 을 선택하여 조인 진행.<br><b>spark.sql.join.preferSortMergeJoin</b> 의 값은 무시됨 |
| spark.sql.adaptive.<br>autoBroadcastJoinThreshold           | 없음   | 브로드캐스트 가능한 데이터프레임의 최대 크기. 이 값을 -1로 설정하면 브로드캐스트는 사용되지 않음. 기본값은 <b>spark.sql.autoBroadcastJoinThreshold</b> 와 동일하며 <b>AQE</b> 가 활성화된 경우에만 사용됨.                                                                                |

# 요약

- Repartition과 Coalesce를 사용한 파티션 크기와 분포 조절
- AQE (Adaptive Query Execution) 소개와 데모



Grepp Inc

한기용

keeyonghan@hotmail.com